

RETT Tax-Strategy Projector

Complete Logic Flow Reference

May 9, 2026 | Post-audit (commit 00cf434)

Confidential — Internal Reference

This document describes the complete end-to-end logic flow of the RETT multi-year tax-strategy projector — how client inputs become projected net-benefit numbers, how multiple strategies are selected and compared, and how supplemental strategies are allocated against the available capital pool.

Stack: Vanilla HTML / CSS / JavaScript — no build step.

Deploy: GitHub Pages — <https://jacobchandler111-svg.github.io/RETT/>

Local: C:\Users\jacob\OneDrive\Desktop\Claude Code\RETT

Latest commit: 00cf434 (Tab 7 full-engine horizon)

Self-test: 307/307 PASS (window.runEngineSelfTest({iterations: 300}))

Table of Contents

- 1 **System Overview**
- 2 **How Strategies Are Selected**
 - 2.1 User Interest (Page 2)
 - 2.2 Auto-Picker: `_autoPickSection()`
 - 2.3 `cfg` Differences Per Strategy
- 3 **The Unified Tax Engine**
 - 3.1 Entry Point
 - 3.2 Mode Detection
 - 3.3 Per-Year Loop (Steps 1–5)
 - 3.4 MetLife Structured-Sale Rules
 - 3.5 Output Row Shape
- 4 **Input Collection and Available Capital**
- 5 **Full Pipeline Orchestration**
 - 5.1 `runFullPipeline()` Execution Order
 - 5.2 Complete Data Flow Diagram
- 6 **Supplemental Strategy Logic**
 - 6.1 Supplemental Registry
 - 6.2 Pre-Meeting Questionnaire Gating
 - 6.3 Rivalry Optimizer (0/1 Knapsack)
 - 6.4 Oil & Gas Multi-Year Deployment

- 7 Why RETT Can Handle Multiple Strategies**
 - 8 Key Global State**
 - 9 Complete Function Index**
-

1. System Overview

RETT is a multi-year tax-strategy projector for Brooklyn capital-loss harvesting. It is built entirely in vanilla HTML, CSS, and JavaScript with no build step — every computation runs client-side in the browser at runtime.

Subsystem Load Order

00-data	→	Tax brackets, Brooklyn fee schedules, state configs
01-brooklyn	→	Loss-curve math, tranche aging, leverage models
02-tax-engine	→	unifiedTaxComparison (core engine, handles A/B/C)
03-solver	→	master-solver, knapsack optimizer, supplemental calcs
05-projections	→	Multi-year projection engine (tranche tracking)
04-ui	→	Form collection, auto-picker, rendering, pipeline

Six-Page Application Structure

Page	ID	Purpose
0	Pre-Meeting (PMQ)	6-question questionnaire; gates supplemental strategies
1	Client Inputs	All inputs → collectInputs() → cfg object
2	Strategies	User marks A/B/C Interested / Not Interested
3	Projection	Auto-picker + engine runs; A/B/C comparison cards
4	Supplemental	Oil & Gas, Delphi, 8 placeholder strategies
5	Strategy Summary	Chosen strategy + funded supplementals + Net Benefit hero

Why RETT Supports Multiple Strategies

Key architectural point: A purely-static HTML calculator (like competing tools) pre-computes all outputs at author-time and displays fixed numbers. RETT's JS solver layer dynamically builds a cfg object per strategy, runs the unified engine, and compares results — enabling real-time optimization that a static file cannot do.

2. How Strategies Are Selected

2.1 User Interest — Page 2

On Page 2, the user clicks Interested or Not Interested for each of the three strategies. The state is stored globally and persisted to localStorage:

```
window.__rettStrategyInterest = {
  A: true | false | null,    // Sell Now
  B: true | false | null,    // Seller Finance ($453 Installment)
  C: true | false | null,    // Structured Sale (MetLife annuity)
}
```

Strategies marked Interested are shown on Page 3. The user then clicks "Use This Strategy" on one card, which sets `window.__rettChosenStrategy = 'A'|'B'|'C'`.

2.2 Auto-Picker: `_autoPickSection(type, baseCfg)`

File: `js/04-ui/projection-dashboard-render.js`

For each interested strategy, the auto-picker sweeps the parameter space to find the configuration that maximizes net benefit. It runs independently for each strategy type before the user chooses one:

Strategy A (Sell Now)

```
for horizon in [1, 2, 4, 5, 6, 7]:
  for shortPct in available_combos:
    score = _scenarioMetrics(cfg)    // net benefit
    if score > best: save (horizon, shortPct)

// In practice: horizon=1 almost always wins (minimizes Brookhaven flat fee)
```

Strategy B (Seller Finance \$453)

```
for horizon in [2, 4, 5, 6, 7]:      // requires horizon >= 2
  for shortPct in available_combos:
    score = _scenarioMetrics(cfg)
    if score > best: save (horizon, shortPct)

// Needs Y0 (recap recognized at ordinary rates) + Y1 (LT gain recognized)
```

Strategy C (Structured Sale)

```
for horizon in [4, 5, 6, 7]:        // requires horizon >= 4
  for duration in _durationsForHorizon(horizon): // [36,48,60,72] filtered
    for r in range(2, min(4, horizon)+1):        // recognition start sweep
      for shortPct in available_combos:
        score = _scenarioMetrics(cfg)
        if score > best: save (horizon, shortPct, duration, r)
```

Helper: `_durationsForHorizon(hor)`

```

availableRecYears = max(0, hor - 1)    // Y0 reserved for sale-year
maxByHor          = availableRecYears * 12
if maxByHor < 36: return []           // MetLife 36-month minimum
return [36, 48, 60, 72].filter(d => d <= maxByHor)

// horizon=4 → recYears=3 → max=36mo → [36]
// horizon=5 → recYears=4 → max=48mo → [36, 48]
// horizon=6 → recYears=5 → max=60mo → [36, 48, 60]
// horizon=7 → recYears=6 → max=72mo → [36, 48, 60, 72]

```

2.3 How cfg Differs Per Strategy

The auto-picker builds a different cfg for each strategy type via `_scenarioCfgFor()`. The table below shows the key fields that differ — these are the engine mode switches:

Field / Aspect	Strategy A Sell Now	Strategy B Seller Finance §453	Strategy C Structured Sale
Sale timing	Close now	Delay to Jan 1 next year	Close now
recognitionStart YearIndex	0 (immediate)	1 (deferred §453)	1–3 (sweep)
maxRecognition YearIndex	null	1 (force all gain Y1)	null (governed by duration)
Brooklyn opens	Year of sale	Jan 1 next year	Year of sale
Horizon minimum	1 year	2 years	4 years
Payment caps	None	None	MetLife rules (see §3.4)
§1250 Recapture timing	Y0, ordinary rates	Y0, ordinary rates (§453(i))	Y0, ordinary rates
LT Gain timing	Y0 (immediate)	Y1 (Jan 1 payment)	Spread Y1 to maturity
Supp deploy years	1	2 (Y0 recap + Y1 LT)	ceil((dur+6)/12)

3. The Unified Tax Engine

3.1 Entry Point

File: `js/02-tax-engine/tax-comparison.js`

```
unifiedTaxComparison(cfg, opts)

Input:  cfg  – complete configuration object (built by _scenarioCfgFor)
        opts – optional overrides (y1LossOverride, etc.)

Output: {
  rows[],           // per-year detail (see §3.5)
  totalBaseline,    // do-nothing tax across horizon
  totalWithStrategy, // tax after Brooklyn losses
  totalSavings,      // baseline - withStrategy
  recognitionSchedule, // gain recognized per year
  ... (loss stats, fee totals)
}
```

3.2 Mode Detection — The Core Switch

The engine handles all three strategies in one function. Strategy routing is determined by a single flag computed at the top of the function:

```
const isDeferred = (cfg.recognitionStartYearIndex || 0) >= 1

isDeferred = false → Strategy A path (immediate, all gain Y0)
isDeferred = true  → Strategy B/C path (deferred, multi-year recognition)

// This is why B and C share one engine: both are deferred.
// They differ only in HOW gain is spread across years.
```

Strategy B forces all gain into a single year (Y1) via `maxRecognitionYearIndex=1`.
 Strategy C spreads gain across multiple years via MetLife payment-schedule caps.
 Both enter the same deferred loop — the caps are what distinguish them.

3.3 Per-Year Loop — Steps 1 Through 5

The engine iterates year *i* from 0 to (`horizonYears - 1`). Each year executes five steps:

Step 1 — Compute Existing-Tranche Loss and Fee


```

for each existing tranche t:
  age      = i - t.startIdx
  yearLoss += t.capital * lossRateForTrancheAge(age)    // Brooklyn loss curve
  yearFee  += t.capital * feeRate * partialYearFactor   // fee pro-rated Y1

// opts.y1LossOverride replaces yearLoss for year 0 if provided

```

Step 2 — Decide Gain to Recognize This Year

```

// Immediate path (Strategy A):
if i === 0: recognize ALL remaining gain now
else:      recognize 0

// Deferred path (Strategy B/C):
maxAbsorbable = Brooklyn's projected loss this year
greedy:  recognize up to maxAbsorbable (absorb as much as Brooklyn can offset)
maturity: force ALL remaining unrecognized gain at maturity year

// MetLife caps (Strategy C only — applied inside deferred path):
Y1:  gainRec ≤ firstPaymentMaxPct * totalGain
     3-year: 40% max   4-year+: 50% max
Y2:  cumulative Y1+Y2 ≤ 80% of totalGain (universal cap)
Final: reserve ≥ lastPaymentMinPct * totalGain
      3-year: 20% min  4-year+: 10% min

```

Step 3 — Carve Tax and Reinvest (Deferred Only)

```

taxCarveOut = gainRecThisYear * _estimateGainTaxRate(cfg)
reinvested  = max(0, gainRecThisYear - taxCarveOut)
reinvested  = min(reinvested, remainingReinvestBudget)

if reinvested > 0:
  push new Brooklyn tranche { capital: reinvested, startIdx: i }

```

Step 4 — Recompute Year Totals With New Tranche

```

yearLoss      = existingLoss + newTrancheLoss
yearFee       = existingFee  + newTrancheFee
yearInvested  = existingInvested + reinvested

```

Step 5 — Apply Brooklyn Losses Against Baseline Tax

Function `_applyLossesWithSTCfCap()` applies accumulated losses against the baseline tax liability in §1(h) priority order — highest tax rate absorbed first:

Loss absorption order (highest-rate bucket first):

1. Short-term gains (ordinary rates – up to 37%)
2. §1250 recapture (25% unrecaptured rate)
3. Long-term gains (0% / 15% / 20%)
4. Ordinary income (capped: \$3,000 MFJ/Single, \$1,500 MFS)

Tracks per year:

- _lossUsed – losses actually applied
- _lossUnused – carryforward to next year
- _ordOffsetApplied – ordinary offset (for state tax adjustments)

3.4 MetLife Structured-Sale Payment Rules

Constants defined at the top of tax-comparison.js (marked TEMPORARY — carrier-negotiated). These enforce the MetLife annuity payment schedule on Strategy C's recognition loop:

Term	Year 1 Max	Y1+Y2 Combined Max	Final Year Min
3-year (36 months)	40% of total gain	80% of total gain	20% of total gain
4-year (48 months)	50% of total gain	80% of total gain	10% of total gain
5-year (60 months)	50% of total gain	80% of total gain	10% of total gain
6-year (72 months)	50% of total gain	80% of total gain	10% of total gain

36 months is the MetLife minimum (floor). 72 months is the practical ceiling.

The auto-picker will not generate an (horizon, duration) pair where $\text{availableRecYears} * 12 < 36$ — those pairs return [] from `_durationsForHorizon()`.

Search METLIFE_RULES or `_isMetLifeConstrained` in the codebase to find every dependent code path.

3.5 Output Row Shape (Per Year)

```
rows[i] = {
  year:                calendar year (year1 + i)
  gainRecognizedThisYear: gain recognized in year i
  lossGenerated:       gross Brooklyn loss generated
  feeCharged:          Brooklyn management fee
  baselineTax:         do-nothing tax (matched-timing for deferred)
  doNothingBaseline:   full Y0 lump-sum baseline (chart only)
  withStrategyTax:     tax after Brooklyn offsets
  savings:            baselineTax - withStrategyTax
  cumSavings:         running sum of savings
  lossUsed:           losses absorbed this year
  lossUnused:         loss carryforward to next year
  ordOffset:          ordinary income offset applied
}
```

4. Input Collection and Available Capital

4.1 collectInputs()

File: js/04-ui/inputs-collector.js — Maps all HTML form fields to a single cfg object that the engine and auto-picker consume.

```

cfg = {
  // Timing
  horizonYears:      5,
  year1:             2026,
  implementationDate: '2026-05-01',
  strategyImplementationDate: '2026-05-01',

  // Tax filing
  filingStatus:      'mfj' | 'single' | 'hoh' | 'mfs',
  state:             'GA' | 'CA' | 'NONE' | ...,

  // Income
  baseOrdinaryIncome: wages + se + biz + rental + dividend + retirement,
  wages:              W-2 + SE (Medicare base only),
  investmentIncomeOrdinary: rental + dividend ($1411 NIIT base),
  baseShortTermGain:  explicit ST gain,

  // Sale
  salePrice:          number,
  costBasis:          number,
  acceleratedDepreciation: number,
  // Derived: LT gain = salePrice - costBasis - acceleratedDepreciation

  // Brooklyn
  custodian:          'schwab' | '',
  tierKey:              'beta1' | ...,
  leverage:           0 to 2.25 (shortPct / 100),
  comboId:            'beta1_200_100' | ...,

  // Engine routing
  recognitionStartYearIndex: 0 | 1 | 2 | 3,
  maxRecognitionYearIndex:  null | 1,
  structuredSaleDurationMonths: 36 (MetLife minimum default),

  // Knobs
  coverTaxesFromSale:  boolean,
  futureSale: { enabled, longTermGain, acceleratedDepreciation }
}

```

4.2 _recomputeAvailableCapital()

File: js/04-ui/controls.js — Derives the pool of capital available for Brooklyn and supplementals from the sale proceeds:

```
availableCapital = salePrice - keepAmount - estimatedTax

keepAmount      = (wantsKeep === 'yes') ? parseUSD(amountField) : 0
estimatedTax     = (wantsCoverTaxes === 'yes') ? _estimatedSaleTax() : 0

_estimatedSaleTax():
  // Conservative-high floor: treats ALL gain as lump-sum Y0 recognition
  // Runs computeFederalTax + computeStateTax on sale-only income
  // Actual deferred tax will be lower (gain is spread; Brooklyn offsets apply)
```

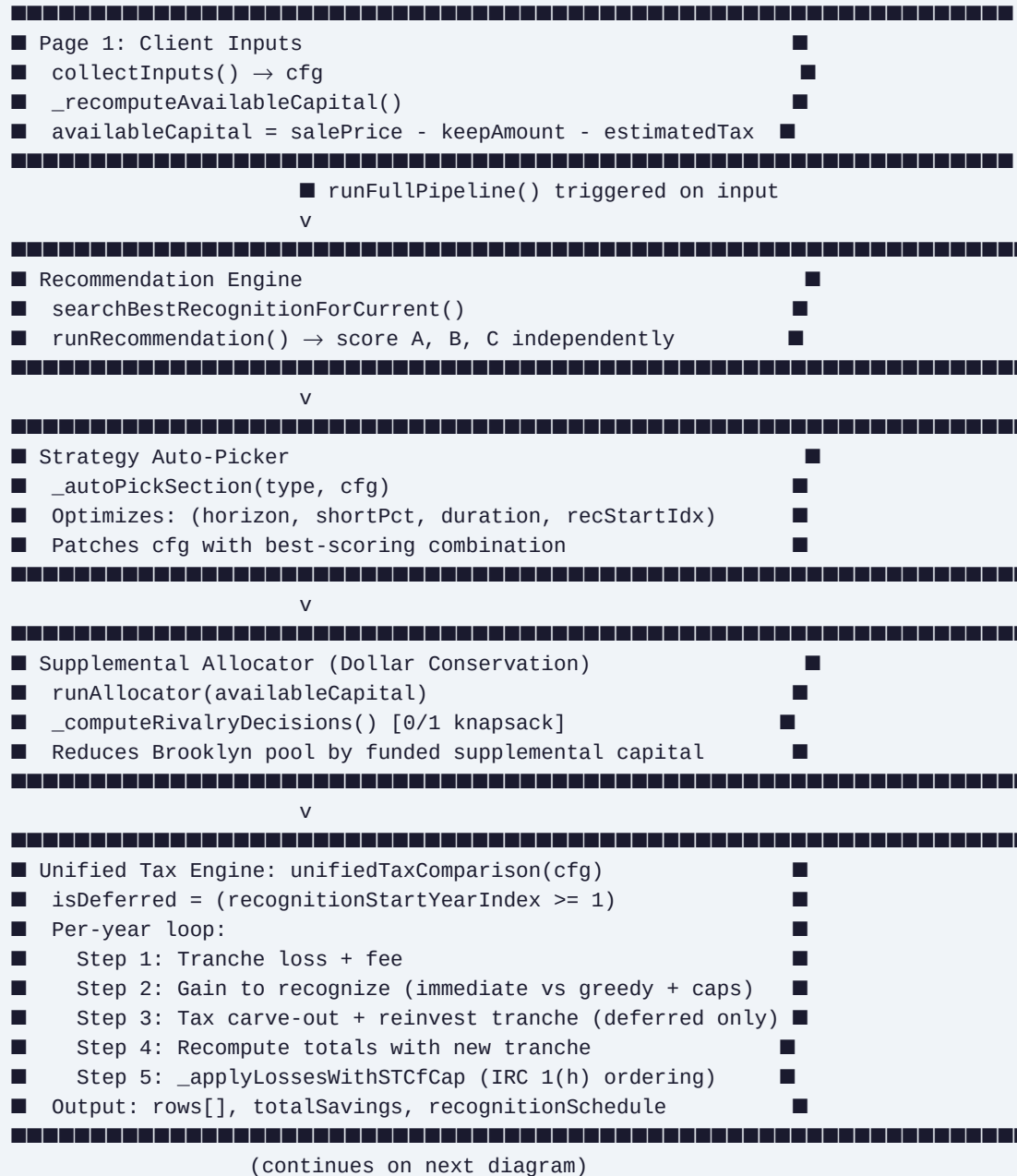
5. Full Pipeline Orchestration

5.1 runFullPipeline() Execution Order

File: js/04-ui/controls.js — Called on every input change.

Step 1	searchBestRecognitionForCurrent() Find optimal recognition-year index for the current cfg.
Step 2	runRecommendation() Compute A/B/C candidates; score each by auto-picker.
Step 3	Strategy Auto-Pick (F20 fix) If __rettAutoPickEnabled AND __rettChosenStrategy is set: apk = _autoPickSection(chosenStrategy, cfg) Patch cfg: horizon, shortPct, combold, recognitionStart, duration
Step 4	Dollar Conservation fundedSuppTotal = runAllocator(availableCapital).allocatedToSupplementals brooklynPool = availableCapital - fundedSuppTotal cfg.investment = min(cfg.investment, brooklynPool)
Step 5	First Projection Pass firstPass = ProjectionEngine.run(cfg) firstLoss = sum of all years' grossLoss
Step 6	Brooklyn Optimizer runBrooklynOptimizer(cfg, firstLoss) If dialBack AND recommendedInvestment < cfg.investment: cfg.investment = recommendedInvestment Run SECOND projection pass at reduced investment
Step 7	Render renderProjectionDashboard() — KPI ribbon, chart, comparison cards

5.2 Complete Data Flow Diagram (Part 1 of 2: Inputs → Engine)



5.2 Complete Data Flow Diagram (Part 2 of 2: Engine → Output)

(continued from previous diagram)

v

- Brooklyn Optimizer: runBrooklynOptimizer(cfg, loss, net) ■
- absorbable = currentLT + currentRecap + futureLT + Recap ■
- If loss > absorbable: scale down investment ■
- Positive-net gate: if net <= 0 then scale = 0 ■
- If dialed back: run second projection pass ■

v

- Page 3: Projection Dashboard ■
- renderProjectionDashboard() - KPI ribbon, chart, cards ■
- buildInterestedSummary() - one card per interested strategy ■
- User clicks 'Use This Strategy' ■
- sets __rettChosenStrategy = 'A' | 'B' | 'C' ■

v

- Page 4: Supplemental Strategies ■
- _yearCountForSaleStrategy(key) - years based on strategy ■
- _oilGasResolvedYears() - per-year {investment,includeRecap} ■
- computeOilGasMultiYear(years) - net benefit + NOL ■
- computeDelphiMultiYear(years, class) - net benefit ■
- User marks each supp Interested / Not Interested ■

v

- Page 5: Strategy Summary ■
- runMasterSolver(primaryNetBenefit) ■
- _computeRivalryDecisions() - which supps are funded ■
- Combined net = primary strategy + funded supplementals ■
- Render: walkaway tiles, Net Benefit hero, Return on Plan ■

6. Supplemental Strategy Logic

6.1 Supplemental Registry

Each supplemental strategy registers itself in `window.__rettSupplementalRegistry`. The solver reads from this registry — it never imports supplemental modules directly.

```

window.__rettSupplementalRegistry['oilGas'] = {
  id:      'oilGas',
  name:    'Oil & Gas Working Interest',
  getInterest: () => window.__rettSupplementalInterest.oilGas,
  getResult:  () => window.__rettOilGasLastResult,
  getNetBenefit:(result) => result.netBenefit,
  getInvestment:(result) => result.totalInvested,
}

// 8 placeholder cards (plan412e3, ptet, qbi, rdCredit,
// plan401h, qcd, solarITC, film181) use __rettSupplementalExtra

```

6.2 Pre-Meeting Questionnaire Gating

PMQ answers gate which supplementals are shown on Page 4. Gating is one-way: answering 'No' auto-sets Not Interested; explicit Interested clicks are never overridden.

Supplemental Strategy	Required PMQ Answer(s)
Oil & Gas, Delphi, Solar ITC, Film 181	altInvestments: Yes
412(e)(3) Plan, 401(h) Plan	businessOwner: Yes
PTET, QBI	businessOwner: Yes AND passThrough: Yes
R&D Credit	businessOwner: Yes AND rdActivity: Yes
QCD (Qualified Charitable Distribution)	charitable: Yes AND age705plus: Yes

6.3 Rivalry Optimizer — 0/1 Knapsack

File: `js/03-solver/master-solver.js`, function `_computeRivalryDecisions(cfg)`

The optimizer decides which supplemental strategies to fund given the available capital pool. It uses an exhaustive 0/1 knapsack over all interested supplementals ($k \leq 20$, so 2^k is tractable at runtime):


```
Objective: maximize SUM(netBenefit_i) - SUM(investment_i) * brooklynRate
Subject to: SUM(investment_i) <= availableCapital
```

```
// Brooklyn's yield rate = netBenefit / investment at full scale
// Any supp that beats Brooklyn's rate is worth funding
```

Decision reasons per supplemental:

```
funded=true, reason='beats-brooklyn':    yield > Brooklyn's rate
funded=true, reason='free-benefit':      investment=0, positive net
funded=false, reason='capital-exhausted': lost to knapsack constraint
funded=false, reason='brooklyn-beats':   yield <= Brooklyn's rate
funded=false, reason='negative-net':     netBenefit <= 0
```

The rivalry optimizer runs TWICE per pipeline: once in Step 4 of runFullPipeline() to compute the Brooklyn pool reduction, and once in runMasterSolver() on Page 5 to render the final combined net benefit with actual funded supplementals shown.

6.4 Oil & Gas Multi-Year Deployment

Files: js/03-solver/calc-oil-gas.js, js/04-ui/supplemental-render.js

```
// Year count by chosen strategy:
Strategy A: 1 year
Strategy B: 2 years
Strategy C: ceil((structuredSaleDurationMonths + 6) / 12)

// Per-year deployment row shape:
{ investment, idcPct: 0.95, includeRecap: (i === 0) }

// Y0 (i=0): includeRecap=true
//   §1250 recapture recognized in sale year at ordinary rates
//   IDC deduction can absorb: ordinary income + recapture income

// Y1+ (i>0): includeRecap=false
//   Per §453(i), recapture is sale-year-only
//   IDC deduction can absorb: ordinary income (wages, W-2) only

// §172 NOL Carryforward (computeOilGasMultiYear):
carryNol = 0
for each year:
  ord_basis = max(0, ord + recap - carryNol) // reduce by prior excess
  if IDC_deduction > ord_basis:
    carryNol = IDC_deduction - ord_basis // excess carries forward
  else:
    carryNol = 0
totalNolGenerated = carryNol (LAST year only – truly wasted residual)
```

7. Why RETT Can Handle Multiple Strategies

The question of why RETT can compare multiple strategies while a competing static HTML calculator cannot comes down to where the computation lives:

Capability	Static HTML Calculator	RETT (JS Engine)
Where computation lives	Author's computer at build time (pre-computed, baked in)	Browser at runtime (computed fresh on each input change)
Multi-strategy comparison	Hard-coded separate sections per strategy; no optimization	<code>_autoPickSection()</code> sweeps parameter space for each strategy type
Parameter optimization	Not possible; author picks one set of values	Sweeps (horizon, shortPct, duration, recognitionStart) — thousands of combos
Input responsiveness	Must reload page or re-publish to change any number	<code>runFullPipeline()</code> re-runs in ms on every form field change
Supplemental rivalry	Not possible; each strategy displayed independently	<code>_computeRivalryDecisions()</code> runs 0/1 knapsack to allocate capital
Tax engine updates	Requires developer to recode and republish static file	Edit one JS file; deploy; engine updates everywhere instantly

Bottom Line

The solver is in JavaScript (not Java), but the architectural principle is the same: runtime computation vs. static display. The JS engine is what enables: (1) dynamic parameter optimization via auto-picker, (2) multi-strategy comparison in one session, (3) supplemental rivalry resolution, and (4) responsive recalculation on any input change — all in milliseconds.

8. Key Global State

The following window globals are shared across all JS modules. They must not be accidentally overwritten — the pipeline reads them at multiple points.

Global	Type	Purpose
__rettStrategyInterest	{ A,B,C: bool null }	Page 2 user interest state per strategy
__rettChosenStrategy	'A' 'B' 'C' null	Set by 'Use This Strategy' button on Page 3
__rettAutoPickEnabled	boolean	true = auto-picker runs; false = manual override
__rettCheckedScenarios	{ A B C: true }	Multi-scenario comparison view on Page 3
__rettSectionState	{ A,B,C: {h,shortPct,...} }	Per-strategy cached auto-pick state
__lastResult	object	Latest ProjectionEngine.run() output
__lastComparison	object	Latest unifiedTaxComparison() output
__rettSupplementalRegistry	{ id: spec }	Supplemental registration map
__rettSupplementalInterest	{ id: bool null }	Page 4 interest state per supplemental
__rettSupplementalEnabled	{ id: bool }	Page 5 toggle state per supplemental
__rettPMQAnswers	{ 6 keys }	Pre-Meeting questionnaire answers (gates supps)
__rettOilGasLastResult	object	Latest Oil & Gas computation result
__rettDelphiLastResult	object	Latest Delphi computation result
__rettSupplementalExtra	{ id: state }	State for 8 placeholder supplemental cards

9. Complete Function Index

js/02-tax-engine/tax-comparison.js

unifiedTaxComparison(cfg, opts)	Main engine entry point — handles all three strategies
_metLifeRulesForTerm(months)	Structured-sale payment caps lookup by term
_buildLossRateByAge(cfg, yf)	Brooklyn loss-curve rate by tranche age
_baseScenarioForYear(cfg, yr, gain, recap)	Per-year tax scenario builder
_yearTaxes(scenario)	Compute federal + state taxes for a scenario
_applyLossesWithSTCfCap(scen, loss, cap)	§1(h) priority-order loss absorption
_structuredSaleMaturityYearIdx(cfg, hor)	Compute maturity year index from duration months
_estimateGainTaxRate(cfg)	Marginal rate for tax carve-out estimate

js/03-solver/master-solver.js

runMasterSolver(primaryNetBenefit)	Combined primary + supplemental view (Page 5)
_computeRivalryDecisions(cfg)	0/1 knapsack: which supplementals to fund
runAllocator(totalAvailable)	Dollar allocation breakdown per funded supp
runBrooklynOptimizer(cfg, loss, net)	Scale Brooklyn investment down if loss exceeds absorbable

js/04-ui/projection-dashboard-render.js

_autoPickSection(type, baseCfg)	Strategy optimizer — sweeps full parameter space
_durationsForHorizon(hor)	Returns feasible structured-sale durations for horizon
_scenarioCfgFor(type, cfg, recC, dur)	Builds engine cfg for a specific strategy type
_scenarioFullData(cfg)	Runs unified + projection engines; returns combined output
buildInterestedSummary()	Page 3 cards — one per interested strategy with optimizer
renderProjectionDashboard()	Renders full Page 3 dashboard

js/04-ui/controls.js

runFullPipeline()	Orchestrates entire data flow (called on every input change)
_recomputeAvailableCapital()	Derives available capital = sale - keep - estimatedTax
_estimatedSaleTax()	Conservative lump-sum tax estimate for capital carve-out

js/04-ui/inputs-collector.js

collectInputs()	Maps all HTML form fields to cfg object
-----------------	---

js/04-ui/supplemental-render.js

_oilGasResolvedYears()	Builds per-year O&G; deployment shape with includeRecap flags
_yearCountForSaleStrategy(key)	Year count for chosen strategy (A=1, B=2, C=ceil)
renderSupplementalPage()	Renders all of Page 4 with supplemental cards
_seedFromSale()	Auto-defaults O&G; / Delphi investment from sale price

js/03-solver/calc-oil-gas.js

computeOilGasMultiYear(years)	Multi-year IDC + §172 NOL carryforward computation
optimizeOilGasMultiYear(inv, meta)	Per-year investment split optimizer

End of RETT Logic Flow Reference — May 2026 — Commit 00cf434